Задание к лабораторной работе №2

«Инференс, декодирование и эффективное дообучение
(маска потерь, LoRA, переобучение, забывание)»

Содержание

1 Описание задания **3**

1.1 Задание 1. Декодирование и проверка маски потерь 6

1.2 Задание 2. Подготовка данных и дообучение LoRA 8

1.3 Задание 3. Переобучение, ранг и катастрофическое забывание 10

1.4 Содержание отчёта и оценивание 13

2 Данные для вариантов **14**

2.1 Вариант 1 16

2.2 Вариант 2 16

2.3 Вариант 3 16

2.4 Вариант 4 17

2.5 Вариант 5 17

2.6 Вариант 6 17

2.7 Вариант 7 18

2.8 Вариант 8 18

2.9 Вариант 9 19

2.10 Вариант 10 19

2.11 Вариант 11 20

2.12 Вариант 12 21

2.13 Вариант 13 22

2.14 Вариант 14 22

2.15 Вариант 15 23

2.16 Вариант 16 23

2.17 Вариант 17 24

2.18 Вариант 18 24

2.19 Вариант 19 25

2.20 Вариант 20 25

2.21 Вариант 21 26

2.22 Вариант 22 26

2.23 Вариант 23 27

2.24 Вариант 24 27

2.25 Вариант 25 28
2.26 Вариант 26 28
2.27 Вариант 27 29
2.28 Вариант 28 29
2.29 Вариант 29 30
2.30 Вариант 30 30

3 Приложения **30**
Приложение А. Генераторы синтетических данных 30
Приложение Б. Метрики задач 31
Приложение В. Версии библиотек 34

1 Описание задания

Цель работы

Освоить инференс и эффективное дообучение генеративной языковой модели: сравнить стратегии декодирования, проверить маску потерь, дообучить модель методом LoRA на задаче своего варианта, рассчитать число обучаемых параметров и память, измерить переобучение, влияние ранга адаптера и катастрофическое забывание.

Модель и окружение

Базовая модель одна для всех вариантов — `TinyLlama/TinyLlama-1.1B-Chat-v1.0` (1,1 млрд параметров). Нужна видеокарта: в fp16 модель занимает около 2,2 ГБ, обучение LoRA — не более 3 ГБ (проверено на видеокарте с 4 ГБ; подойдёт бесплатный Google Colab с T4). Если памяти не хватает, загрузите модель в 4 бита: `load_base(four_bit=True)`. Библиотеки — приложение В.

Модель обучена в формате чата; текст диалога строится функцией `tokenizer.apply_chat_template` и выглядит так:

```
<|user|>
{instruction}</s>
<|assistant|>
{response}</s>
```

Общие требования

- Номер варианта и зерно генератора случайных чисел задаются в первой ячейке; `SEED = 1000 + VARIANT`.
- Сравнения «базовая модель → после дообучения» выполняются на одних и тех же тестовых примерах (`test_pairs`).
- Отчёт — исполненный ноутбук в формате «код → таблица или график → вывод»; вывод — 3–5 предложений с числами.

Первая ячейка ноутбука: загрузка модели и функции для формата чата и генерации.

```
VARIANT = 1                    # номер своего варианта, 1..30
SEED = 1000 + VARIANT
```

```
N_TRAIN = 1000                  # обучающих пар инструкция« -> ответ»
N_EVAL = 50                     # тестовых примеров для метрики задачи

import json, math, random, re, time
import numpy as np
import torch
from datasets import load_dataset, Dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig

random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

MODEL_NAME = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)


def load_base(four_bit=False):
    """fp16 занимает ~2.2 ГБ видеопамяти; four_bit=True (bitsandbytes) -- ~0.8 ГБ."""
    q = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                           bnb_4bit_compute_dtype=torch.float16) if four_bit else None
    return AutoModelForCausalLM.from_pretrained(
        MODEL_NAME, dtype=torch.float16, device_map={"": 0}, quantization_config=q)


def chat(instruction, response=None):
    """Текст в формате чата модели: <|user|>\\n...</s>\\n<|assistant|>\\n..."""
    msgs = [{"role": "user", "content": instruction}]
    if response is None:
        return tokenizer.apply_chat_template(msgs, tokenize=False,
    add_generation_prompt=True)
    msgs.append({"role": "assistant", "content": response})
    return tokenizer.apply_chat_template(msgs, tokenize=False)


def cut(text, n=512):
    """Обрезает длинный входной документ до n токенов, чтобы пример помещался в
    max_length."""
    ids = tokenizer(text, add_special_tokens=False)["input_ids"]
    return text if len(ids) <= n else tokenizer.decode(ids[:n])


@torch.no_grad()
def generate(model, instructions, max_new_tokens=100, **gen):
    model.eval()
    outs = []
    for s in instructions:
        enc = tokenizer(chat(s), return_tensors="pt", add_special_tokens=False,
```

```
                        truncation=True, max_length=1536).to(model.device)
        o = model.generate(**enc, max_new_tokens=max_new_tokens,
                           pad_token_id=tokenizer.eos_token_id, **gen)
        outs.append(tokenizer.decode(o[0, enc["input_ids"].shape[1]:],
                                     skip_special_tokens=True).strip())
    return outs


base = load_base()
```

Далее выполняются функции метрик (приложение Б) и код своего варианта (раздел 2). Код варианта задаёт `train_raw`, `test_raw`, функцию `to_pair(ex)`, которая превращает пример датасета в пару «инструкция, эталонный ответ», метрику задачи `metric(P, R)` (список ответов модели, список эталонов), `MAX_NEW_TOKENS` и ряд рангов `RANKS`. После него готовятся данные (шаг 2.1):

```
# train_raw, test_raw, to_pair заданы кодом варианта раздел( 2)
tr = train_raw.shuffle(seed=SEED).select(range(min(N_TRAIN + 100, len(train_raw))))
pairs = [to_pair(ex) for ex in tr]
train_pairs, val_pairs = pairs[:-100], pairs[-100:]
test_raw_s = test_raw.shuffle(seed=SEED)
test_pairs = [to_pair(ex) for ex in test_raw_s.select(range(min(100, len(test_raw_s))))]


def to_records(pairs):
    """Три поля: prompt, response и маска (0 на токенах промпта, 1 на токенах ответа)."""
    rec = []
    for p, r in pairs:
        n_p = len(tokenizer(chat(p), add_special_tokens=False)["input_ids"])
        n_all = len(tokenizer(chat(p, r), add_special_tokens=False)["input_ids"])
        rec.append({"prompt": p, "response": r, "mask": [0] * n_p + [1] * (n_all - n_p)})
    return rec


records = to_records(train_pairs)


def to_sft(pairs):
    """Формат prompt/completion: SFTTrainer сам применит шаблон чата и маску
    (completion_only_loss)."""
    return Dataset.from_list([{"prompt": [{"role": "user", "content": p}],
                               "completion": [{"role": "assistant", "content": r}]}
                              for p, r in pairs])
```

```
train_sft, val_sft = to_sft(train_pairs), to_sft(val_pairs)
```

1.1 Задание 1. Декодирование и проверка маски потерь

Шаг 1.1. Стратегии декодирования. Сгенерируйте базовой моделью ответы на 10 тестовых инструкций в пяти режимах (`max_new_tokens=100`). Для каждого режима посчитайте Distinct-2 (доля уникальных биграмм слов), долю повторных 3-грамм и среднюю длину ответа в словах. Найдите среди ответов по одному примеру зацикливания и галлюцинации (утверждения, которого нет во входных данных).

```
MODES = {
    "Greedy":      dict(do_sample=False),
    "Temperature": dict(do_sample=True, temperature=0.7),
    "Top-k":       dict(do_sample=True, top_k=40, temperature=0.7),
    "Top-p":       dict(do_sample=True, top_p=0.9, temperature=0.7),
    "Beam":        dict(do_sample=False, num_beams=4),
}


def repetition_rate(texts):
    """Доля повторных триграмм слов внутри ответа, среднее по ответам."""
    r = []
    for t in texts:
        w = t.split(); tri = [tuple(w[i:i + 3]) for i in range(len(w) - 2)]
        r.append(1 - len(set(tri)) / len(tri) if tri else 0.0)
    return float(np.mean(r))


prompts10 = [x for x, _ in test_pairs[:10]]
for name, g in MODES.items():
    outs = generate(base, prompts10, max_new_tokens=100, **g)
    print(f"{name:12} Distinct-2={distinct2(outs):.3f} Rep={repetition_rate(outs):.3f} "
          f"длина={np.mean([len(o.split()) for o in outs]):.1f}")
```

Материал: лекция 3 (декодирование, температура, top-k и top-p).

Шаг 1.2. Маска потерь. Разберите пример: промпт «Извлеки город: рейс прилетел в Казань 12 мая», ответ `{"город": "Казань"дата": "12.05"}`; лента из 10 токенов, позиции 1–6 — промпт (маска $m_t = 0$), 7–10 — ответ ($m_t = 1$); сумма потерь на промпте 3.6, на ответе 4.0.

1. Посчитайте потерю примера без маски и с маской: без маски — сумма всех потерь, делённая на число всех токенов, $(3.6+4.0)/10 = 0.76$; с маской — сумма потерь ответа, делённая на число токенов ответа, $4.0/4 = 1.00$. Объясните, почему эти два числа нельзя сравнивать напрямую (нормировки разные).
2. Соберите на токенизаторе модели пару «промпт + ответ» первого обучающего примера и функцией `build_labels` постройте метки. Выведите текстом токены, у которых метка не равна -100, и убедитесь, что это ровно токены ответа.
3. Посчитайте на базовой модели потерю этого примера без маски и с маской.
4. Покажите следствие сдвига меток: логит позиции t предсказывает токен $t+1$, поэтому последний токен промпта уже участвует в потере — по нему модель учится выдавать первый токен ответа. Граница маски в векторе потерь сдвинута на одну позицию влево относительно границы в `labels`.

```
def build_labels(input_ids, prompt_len):
    labels = input_ids.clone()          # сдвиг меток (label shift) делает сама модель
    внутри loss
    labels[:prompt_len] = -100          # позиции промпта не участвуют в потере
    return labels



instr, resp = train_pairs[0]
full_ids = tokenizer(chat(instr, resp), add_special_tokens=False,
    return_tensors="pt")["input_ids"][0]
prompt_len = len(tokenizer(chat(instr), add_special_tokens=False)["input_ids"])
labels = build_labels(full_ids, prompt_len)
print("m_t = 1 на токенах:", repr(tokenizer.decode(full_ids[labels != -100])))

# потери по позициям: logits[t] предсказывает токен t+1
with torch.no_grad():
    logits = base(full_ids[None].to(base.device)).logits[0, :-1].float()
tok_loss = torch.nn.functional.cross_entropy(logits, full_ids[1:].to(base.device),
    reduction="none")
m = (labels[1:] != -100).to(base.device)        # маска для предсказываемых токенов
print("6ез маски:", tok_loss.mean().item())
print("с маской :", tok_loss[m].mean().item())
```

Материал: лекция 4, слайды «Пример данных: токены и маска» и «Маска потерь: поточечная таблица».

Что сдать: таблица метрик декодирования для пяти режимов, два примера сбоев, вывод токенов ответа и сравнение потерь без маски и с маской (пример из задания и свой).

1.2 Задание 2. Подготовка данных и дообучение LoRA

Шаг 2.1. Данные. Код подготовки данных выше берёт `N_TRAIN = 1000` обучающих пар и 100 валидационных (в варианте 26 обучающих пар 800). Покажите для двух примеров три поля: `prompt`, `response` и `mask` (список `records`).

Шаг 2.2. Число параметров LoRA. Для матрицы весов размера $d \times k$ адаптер ранга r содержит $r(d + k)$ параметров (матрицы A размера $d \times r$ и B размера $r \times k$). Пример для Llama-7B: $d = k = 4096$, $r = 8 \to 65\,536$ параметров на матрицу ($\approx 0.39\%$ матрицы), 6 матриц $\times$ 32 слоя $\to$ 12,6 млн ($\approx 0.18\%$ модели).

Проделайте тот же расчёт для своей модели и первого ранга своего ряда: число параметров на каждую адаптируемую матрицу, итог по всем слоям, доля от всех параметров модели. Размеры d и k берите из формы каждой матрицы, а не из `hidden_size`: в TinyLlama используется Grouped-Query Attention — у ключей и значений 4 головы вместо 32, поэтому `v_proj` имеет размер 2048×256, а не 2048×2048. Расчёт оформите таблицей.

```
cfg = base.config
print("hidden_size:", cfg.hidden_size, "слоёв:", cfg.num_hidden_layers,
      "голов Q:", cfg.num_attention_heads, "голов K/V:", cfg.num_key_value_heads)
attn = base.model.layers[0].self_attn
r = RANKS[0]
total = 0
for name in ("q_proj", "v_proj"):
    W = getattr(attn, name).weight            # форма [k, d]: выход k, вход d
    k, d = W.shape
    per = r * (d + k)
    total += per * cfg.num_hidden_layers
    print(f"{name}: d={d}, k={k}, r(d+k) = {per}")
n_all = sum(p.numel() for p in base.parameters())
print(f"итого LoRA: {total}  ({100 * total / n_all:.3f}% от {n_all})")
```

Материал: лекция 4, раздел о низкоранговом дообучении (формула числа параметров LoRA).

Шаги 2.3–2.4. Настройка и обучение LoRA. Адаптеры ставятся на матрицы W_Q и W_V (экономный набор). Масштаб обновления адаптера равен α/r; чтобы эксперименты с разными рангами были сравнимы, α и скорость обучения не меняются: $\alpha = 16$ для всех рангов. Обучение — `SFTTrainer` из библиотеки `trl`: до 4 эпох, скорость обучения $2 \cdot 10^{-4}$, батч 2 с накоплением градиента 8 шагов (эффективный батч 16), валидация после каждой эпохи, ранняя остановка с терпением 1 эпоха. Маска потерь строится автоматически (`completion_only_loss=True`): потеря считается только по токенам ответа. Сравните вывод `print_trainable_parameters()` со своим расчётом шага 2.2.

```
from peft import LoraConfig, TaskType
from trl import SFTTrainer, SFTConfig
from transformers import EarlyStoppingCallback


def train_lora(rank, epochs=4, out_dir="./lora"):
    model = load_base()
    lora = LoraConfig(r=rank, lora_alpha=16,           # alpha одинаков для всех рангов
                      target_modules=["q_proj", "v_proj"],
                      lora_dropout=0.05, bias="none", task_type=TaskType.CAUSAL_LM)
    args = SFTConfig(
        output_dir=f"{out_dir}_r{rank}", num_train_epochs=epochs,
        per_device_train_batch_size=2, gradient_accumulation_steps=8,
        learning_rate=2e-4, max_length=1024, completion_only_loss=True,
        eval_strategy="epoch", save_strategy="epoch", logging_strategy="epoch",
        load_best_model_at_end=True, metric_for_best_model="eval_loss",
        save_total_limit=1, gradient_checkpointing=True, fp16=True,
        seed=SEED, report_to="none")
    trainer = SFTTrainer(model=model, args=args, train_dataset=train_sft,
                         eval_dataset=val_sft, processing_class=tokenizer,
                         peft_config=lora,
                         callbacks=[EarlyStoppingCallback(early_stopping_patience=1)])
    trainer.model.print_trainable_parameters()
    trainer.train()
    return trainer


import gc
del base; gc.collect(); torch.cuda.empty_cache()     # две копии модели не помещаются в
```

```
    4 ГБ: база загрузится в train_lora
t0 = time.time()
trainer = train_lora(RANKS[0])
t_train = time.time() - t0
hist = trainer.state.log_history
train_loss = [h["loss"] for h in hist if "loss" in h]
val_loss = [h["eval_loss"] for h in hist if "eval_loss" in h]
```

Материал: лекция 4 (масштаб α/r, инициализация адаптера: A — случайная, B — нули).

Шаг 2.5. Контрольный вывод. Сгенерируйте ответы на тестовые инструкции базовой моделью и моделью с адаптером (перед обучением базовая модель удаляется из видеопамяти, поэтому её ответы получают из той же модели с выключенным адаптером — `model.disable_adapter()`), посчитайте метрику задачи до и после обучения на `N_EVAL = 50` примерах. Приведите 5 пар ответов «до/после». Соблюдает ли модель после обучения формат ответа? Модель, обученная без маски, обычно повторяет промпт; с маской — выдаёт ответ в нужном формате.

```
instr = [x for x, _ in test_pairs[:N_EVAL]]
refs = [y for _, y in test_pairs[:N_EVAL]]
model = trainer.model                              # базовая модель + адаптер LoRA
with model.disable_adapter():                      # адаптер выключен -> ответы базовой
    модели
    before = generate(model, instr, MAX_NEW_TOKENS)
after = generate(model, instr, MAX_NEW_TOKENS)
print("метрика до   :", metric(before, refs))
print("метрика после:", metric(after, refs))
```

Что сдать: таблица расчёта параметров и сверка с `print_trainable_parameters()`, метрика до и после обучения, 5 пар ответов.

1.3 Задание 3. Переобучение, ранг и катастрофическое забывание

Шаг 3.1. Кривые переобучения. Постройте график train loss и val loss по эпохам. Ответьте на вопросы: на какой эпохе минимум val loss; виден ли разрыв «val растёт, train падает»; на какой эпохе сработала (или сработала бы) ранняя остановка с терпением 1. Заполните таблицу трассировки ранней остановки (Best — лучшая val loss на данный момент, Wait — число эпох без улучшения):

Эпоха	Val loss	Best	Wait	Действие
1	…	…	0	сохранить
2	…	…	…	…
…	…	…	…	…

Шаг 3.2. Катастрофическое забывание. Дообучение на узкой задаче может ухудшить общие знания модели; это измеряют перплексией на общем корпусе. Пример: 14.2 у базовой модели, 22.8 после полного дообучения и 15.1 после LoRA. Измерьте перплексию $PP = \exp(\text{средняя кросс-энтропия на токен})$ на 50 отрезках по 512 токенов корпуса wikitext-2 до и после дообучения и заполните таблицу:

Метрика	База	После LoRA	Изменение
Метрика задачи варианта	…	…	…
Перплексия общего корпуса	…	…	…

Сформулируйте вывод: насколько LoRA сохранила знания базовой модели (в примере рост перплексии у LoRA 0.9 против 8.6 у полного дообучения). Абсолютные значения перплексии зависят от модели и корпуса — сравнивайте изменения.

```
wiki = load_dataset("Salesforce/wikitext", "wikitext-2-raw-v1")["test"]
wiki_ids = tokenizer("\n\n".join(wiki["text"]), return_tensors="pt")["input_ids"][0]



@torch.no_grad()
def perplexity(model, n_seg=50, seg_len=512):
    """PP = expсредняя( cross_entropy на токен) по n_seg отрезкам wikitext."""
    model.eval()
    nll, n = 0.0, 0
    for i in range(n_seg):
        seg = wiki_ids[i * seg_len:(i + 1) * seg_len][None].to(model.device)
        loss = model(seg, labels=seg).loss.item()
        nll += loss * (seg.shape[1] - 1); n += seg.shape[1] - 1
    return math.exp(nll / n)



with model.disable_adapter():
    pp_base = perplexity(model)
```

```
pp_lora = perplexity(model)
print(f"PP база: {pp_base:.2f}   PP LoRA: {pp_lora:.2f}   изменение: {pp_lora -
    pp_base:+.2f}")
```

Материал: лекция 4, разделы о переобучении, ранней остановке и катастрофическом забывании.

Шаг 3.3. Память по правилу «18 байт на параметр». При обучении со смешанной точностью за каждый обучаемый параметр платится мастер-вес 4 Б + градиент 4 Б + два момента Adam 8 Б + рабочий вес 2 Б = 18 Б. Оцените теоретическую память (без активаций) полного дообучения своей модели и дообучения LoRA (обучаемые параметры $\times$ 18 Б + хранение базы по 2 Б на параметр). Сравните с фактическим пиком `torch.cuda.max_memory_allocated()` и объясните расхождение.

```
BYTES_PER_TRAINABLE = 4 + 4 + 8 + 2     # основной вес fp32 + градиент + два момента
    Adam + рабочий вес fp16
n_lora = sum(p.numel() for p in model.parameters() if p.requires_grad)
n_all = sum(p.numel() for p in model.parameters()) - n_lora     # параметры базовой
    модели
full_ft = n_all * BYTES_PER_TRAINABLE
lora_mem = n_lora * BYTES_PER_TRAINABLE + n_all * 2   # база хранится в fp16 и не
    обучается
print(f"полное дообучение: {full_ft / 2**30:.1f} ГиБ,  LoRA: {lora_mem / 2**30:.2f} ГиБ
    без( активаций)")
if torch.cuda.is_available():
    print(f"факт пик(): {torch.cuda.max_memory_allocated() / 2**30:.2f} ГиБ")
```

Шаг 3.4. Влияние ранга (основной эксперимент). Ранг работает как регуляризатор: уменьшая r, модель осознанно недообучают в деталях ради обобщения. Обучите адаптеры для всех рангов своего ряда `RANKS` (первый уже обучен в задании 2, его результаты берутся из `trainer`; перед циклом модель удаляется из видеопамяти, поэтому этот шаг выполняется последним) с одинаковыми остальными настройками. Для каждого ранга приведите число обучаемых параметров, лучшую val loss, метрику задачи и время обучения. Постройте график «метрика от ранга» и найдите точку насыщения.

```
results = {RANKS[0]: {"params": n_lora, "val_loss": trainer.state.best_metric,
                      "metric": metric(after, refs), "время, с": round(t_train)}}
del trainer, model; gc.collect(); torch.cuda.empty_cache()      # освобождаем
    видеопамять под следующие обучения
for r in RANKS[1:]:
```

```
    t0 = time.time()
    t = train_lora(r)
    P = generate(t.model, instr, MAX_NEW_TOKENS)
    results[r] = {"params": sum(p.numel() for p in t.model.parameters() if
    p.requires_grad),
                  "val_loss": t.state.best_metric, "metric": metric(P, refs),
                  "время, с": round(time.time() - t0)}
    del t, P; gc.collect(); torch.cuda.empty_cache()
print(results)
```

Шаг 3.5. Диагностика сбоев. Отметьте в таблице симптомы, которые вы наблюдали в своих экспериментах, и приведите примеры:

Симптом	Причина	Действие
Модель повторяет промпт	маска не на ответе	$m_t = 1$ только на токенах ответа
train↓, val↑ со 2-й эпохи	переобучение	ранняя остановка, меньший r
Старые умения пропали	полное дообучение без replay	replay 10–20% старых данных, LoRA
Потеря не снижается	B инициализирована не нулями, мала скорость обучения	проверить инициализацию и скорость обучения
«Двойной» вывод после слияния	адаптер слит дважды	пересобрать из базовой модели
NaN в потере	скорость обучения велика для fp16	снизить скорость обучения, клиппинг градиента

Материал: лекция 4 (полное, низкоранговое и квантованное дообучение; таблица диагностики сбоев).

Что сдать: график переобучения и таблица трассировки, таблица забывания, расчёт памяти, таблица и график по рангам, отмеченная таблица диагностики.

1.4 Содержание отчёта и оценивание

Отчёт — исполненный ноутбук: все ячейки выполнены, графики подписаны (оси, легенда), после каждого задания — вывод с числами. Обязательны: проверка маски потерь, ручной расчёт параметров со сверкой, таблица

трассировки ранней остановки, таблица рангов, таблица забывания, расчёт памяти, диагностика сбоев.

Критерий	Вес
Задание 1: декодирование и проверка маски	25%
Задание 2: расчёт параметров, модель дообучена, сравнение до/после	30%
Задание 3: переобучение, ранги, забывание, память, диагностика	30%
Оформление и качество выводов	15%

2 Данные для вариантов

Таблица 1: Варианты заданий (ряд рангов — значения r для шага 3.4)

№	Тип задачи	Датасет	Метрика	Ряд рангов r
1	Суммаризация	abisee/cnn_dailymail	ROUGE-L	4, 8, 16
2	Суммаризация	abisee/cnn_dailymail	ROUGE-1	8, 16, 32
3	Суммаризация	EdinburghNLP/xsum	ROUGE-L	4, 8, 16
4	Суммаризация	EdinburghNLP/xsum	ROUGE-2	8, 16, 32
5	Суммаризация	knkarthick/samsum	ROUGE-L	4, 8, 16
6	Инструкции	databricks/databricks-dolly-15k	ROUGE-L, длина	4, 8, 16
7	Инструкции	yahma/alpaca-cleaned	ROUGE-L	8, 16, 32
8	Вопрос по тексту	dolly-15k, closed_qa	Token-F1	4, 8, 16
9	Инструкции	alpaca-cleaned, с input	ROUGE-L	16, 32, 64
10	Суммаризация	dolly-15k, summarization	ROUGE-L	8, 16, 32
11	Извлечение сущностей	eriktks/conll2003	Entity-F1	4, 8, 16
12	Извлечение сущностей	eriktks/conll2003	Recall сущностей	8, 16, 32
13	Вопрос-ответ	rajpurkar/squad	Exact Match, F1	4, 8, 16
14	Вопрос-ответ	rajpurkar/squad_v2	F1, «нет ответа»	8, 16, 32
15	Вопрос-ответ	rajpurkar/squad, числа	F1	16, 32, 64
16	Перевод	opus-100, en→de	BLEU	4, 8, 16

№	Тип задачи	Датасет	Метрика	Ряд рангов r
17	Перевод	opus-100, en→fr	BLEU	8, 16, 32
18	Перефразирование	nyu-mll/glue, qqp	Сем. близость	4, 8, 16
19	Перефразирование	nyu-mll/glue, qqp	Distinct-2, ROUGE-L	8, 16, 32
20	Перевод	opus-100, en→ru	BLEU, длина	16, 32, 64
21	Классификация генерацией	fancyzhx/ag_news	Точность метки	4, 8, 16
22	Классификация генерацией	dair-ai/emotion	Точность метки	8, 16, 32
23	Классификация генерацией	cardiffnlp/tweet_eval	Точность метки	4, 8, 16
24	Рассуждение	openai/gsm8k	Формат, точность	8, 16, 32
25	Структурный вывод	синтетика (прил. А)	Точность полей	4, 8, 16
26	Медицина	qiaojin/PubMedQA	Точность	8, 16, 32
27	Финансы	financial_phrasebank	Точность метки	4, 8, 16
28	Стилизация	dolly-15k	Соблюдение стиля	8, 16, 32
29	Структурный вывод	синтетика (прил. А)	Точность полей	4, 8, 16
30	Суммаризация	knkarthick/dialogsum	ROUGE-L	8, 16, 32, 64

Код варианта выполняется после первой ячейки и функций метрик (приложение Б). Датасеты загружаются с Hugging Face Hub по полному имени `автор/название`; для датасетов, которые в оригинале загружаются только скриптом или удалены, указаны копии.

2.1 Вариант 1

Дано: суммаризация. Краткое изложение новости в 3–4 предложениях; статья обрезается до 512 токенов. Метрика: ROUGE-L. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("abisee/cnn_dailymail", "3.0.0")
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Summarize the news article in 3-4 sentences.\n\n" +
    cut(ex["article"]), ex["highlights"])
metric = rouge("rougeL")
MAX_NEW_TOKENS = 128
RANKS = [4, 8, 16]
```

2.2 Вариант 2

Дано: суммаризация. Краткое изложение новости в 3–4 предложениях; статья обрезается до 512 токенов. Метрика: ROUGE-1. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("abisee/cnn_dailymail", "3.0.0")
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Summarize the news article in 3-4 sentences.\n\n" +
    cut(ex["article"]), ex["highlights"])
metric = rouge("rouge1")
MAX_NEW_TOKENS = 128
RANKS = [8, 16, 32]
```

2.3 Вариант 3

Дано: суммаризация. Изложение статьи BBC одним предложением; статья обрезается до 512 токенов. Метрика: ROUGE-L. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("EdinburghNLP/xsum")
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Summarize the article in one sentence.\n\n" +
    cut(ex["document"]), ex["summary"])
metric = rouge("rougeL")
MAX_NEW_TOKENS = 64
RANKS = [4, 8, 16]
```

2.4 Вариант 4

Дано: суммаризация. Изложение статьи BBC одним предложением; статья обрезается до 512 токенов. Метрика: ROUGE-2. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("EdinburghNLP/xsum")
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Summarize the article in one sentence.\n\n" +
    cut(ex["document"]), ex["summary"])
metric = rouge("rouge2")
MAX_NEW_TOKENS = 64
RANKS = [8, 16, 32]
```

2.5 Вариант 5

Дано: суммаризация. Краткое изложение переписки в мессенджере (SAMSum). Метрика: ROUGE-L. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("knkarthick/samsum")          # копия SAMSum: оригинал удалён с Hub
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Summarize the dialogue.\n\n" + ex["dialogue"], ex["summary"])
metric = rouge("rougeL")
MAX_NEW_TOKENS = 96
RANKS = [4, 8, 16]
```

2.6 Вариант 6

Дано: инструкции. Ответы на инструкции всех категорий Dolly; дополнительно — отношение длины ответа к длине эталона. Метрика: ROUGE-L, длина. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("databricks/databricks-dolly-15k")["train"]
sp = ds.train_test_split(test_size=0.1, seed=SEED)
train_raw, test_raw = sp["train"], sp["test"]
to_pair = lambda ex: (ex["instruction"] + ("\n\n" + ex["context"] if ex["context"] else
    ""), ex["response"])
metric = rouge("rougeL")   # дополнительно: length_ratio(P, R)
MAX_NEW_TOKENS = 256
RANKS = [4, 8, 16]
```

2.7 Вариант 7

Дано: инструкции. Ответы на инструкции Alpaca. Метрика: ROUGE-L. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("yahma/alpaca-cleaned")["train"]
sp = ds.train_test_split(test_size=0.05, seed=SEED)
train_raw, test_raw = sp["train"], sp["test"]
to_pair = lambda ex: (ex["instruction"] + ("\n\n" + ex["input"] if ex["input"] else ""),
    ex["output"])
metric = rouge("rougeL")
MAX_NEW_TOKENS = 256
RANKS = [8, 16, 32]
```

2.8 Вариант 8

Дано: вопрос по тексту. Ответ на вопрос по приложенному тексту (категория closed_qa); текст обрезается до 512 токенов. Метрика: Token-F1. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("databricks/databricks-dolly-15k")["train"]
ds = ds.filter(lambda ex: ex["category"] == "closed_qa")
sp = ds.train_test_split(test_size=0.1, seed=SEED)
train_raw, test_raw = sp["train"], sp["test"]
to_pair = lambda ex: (ex["instruction"] + ("\n\n" + cut(ex["context"]) if ex["context"]
    else ""), ex["response"])
metric = squad_f1
MAX_NEW_TOKENS = 64
RANKS = [4, 8, 16]
```

2.9 Вариант 9

Дано: инструкции. Инструкции Alpaca, у которых есть входные данные (поле input). Метрика: ROUGE-L. Ряд рангов: 16, 32, 64.

```
ds = load_dataset("yahma/alpaca-cleaned")["train"]
ds = ds.filter(lambda ex: bool(ex["input"]))        # только инструкции с входными
    данными
sp = ds.train_test_split(test_size=0.05, seed=SEED)
train_raw, test_raw = sp["train"], sp["test"]
to_pair = lambda ex: (ex["instruction"] + ("\n\n" + ex["input"] if ex["input"] else ""),
    ex["output"])
metric = rouge("rougeL")
MAX_NEW_TOKENS = 256
RANKS = [16, 32, 64]
```

2.10 Вариант 10

Дано: суммаризация. Краткое изложение приложенного текста (категория summarization, около 1200 примеров); текст обрезается до 512 токенов. Метрика: ROUGE-L. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("databricks/databricks-dolly-15k")["train"]
ds = ds.filter(lambda ex: ex["category"] == "summarization")
sp = ds.train_test_split(test_size=0.1, seed=SEED)
train_raw, test_raw = sp["train"], sp["test"]
to_pair = lambda ex: (ex["instruction"] + ("\n\n" + cut(ex["context"]) if ex["context"]
    else ""), ex["response"])
metric = rouge("rougeL")
MAX_NEW_TOKENS = 128
RANKS = [8, 16, 32]
```

2.11 Вариант 11

Дано: извлечение сущностей. Список именованных сущностей в формате `PER: ...; LOC: ...`. Метрика: Entity-F1. Ряд рангов: 4, 8, 16.

```
# оригинал загружается только скриптом; берём его копию в формате parquet
ds = load_dataset("eriktks/conll2003", revision="refs/convert/parquet")
train_raw, test_raw = ds["train"], ds["test"]
TAGS = ds["train"].features["ner_tags"].feature.names    # O, B-PER, I-PER, B-ORG, ...


def to_pair(ex):
    ents, cur = {}, None
    for w, t in zip(ex["tokens"], ex["ner_tags"]):
        tag = TAGS[t]
        if tag.startswith("B-"):
            cur = [w]; ents.setdefault(tag[2:], []).append(cur)
        elif tag.startswith("I-") and cur is not None:
            cur.append(w)
        else:
            cur = None
    answer = "; ".join(k + ": " + ", ".join(" ".join(x) for x in v) for k, v in
    sorted(ents.items()))
    return ("Extract named entities (PER, ORG, LOC, MISC) from the sentence.\n\n"
            + " ".join(ex["tokens"]), answer or "none")


metric = lambda P, R: entity_prf(P, R)["f1"]
MAX_NEW_TOKENS = 64
RANKS = [4, 8, 16]
```

2.12 Вариант 12

Дано: извлечение сущностей. Список именованных сущностей; метрика — полнота (доля найденных эталонных сущностей). Метрика: Recall сущностей. Ряд рангов: 8, 16, 32.

```
# оригинал загружается только скриптом; берём его копию в формате parquet
ds = load_dataset("eriktks/conll2003", revision="refs/convert/parquet")
train_raw, test_raw = ds["train"], ds["test"]
TAGS = ds["train"].features["ner_tags"].feature.names    # O, B-PER, I-PER, B-ORG, ...


def to_pair(ex):
    ents, cur = {}, None
    for w, t in zip(ex["tokens"], ex["ner_tags"]):
        tag = TAGS[t]
        if tag.startswith("B-"):
            cur = [w]; ents.setdefault(tag[2:], []).append(cur)
        elif tag.startswith("I-") and cur is not None:
            cur.append(w)
        else:
            cur = None
    answer = "; ".join(k + ": " + ", ".join(" ".join(x) for x in v) for k, v in
    sorted(ents.items()))
    return ("Extract named entities (PER, ORG, LOC, MISC) from the sentence.\n\n"
            + " ".join(ex["tokens"]), answer or "none")


metric = lambda P, R: entity_prf(P, R)["recall"]
MAX_NEW_TOKENS = 64
RANKS = [8, 16, 32]
```

2.13 Вариант 13

Дано: вопрос-ответ. Ответ на вопрос фрагментом текста; основная метрика — точное совпадение, дополнительная — F1 по словам. Метрика: Exact Match, F1. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("rajpurkar/squad")
train_raw, test_raw = ds["train"], ds["validation"]
to_pair = lambda ex: ("Answer the question using the context. If there is no answer, say
    'no answer'.\n\n"
                      f"Context: {ex['context']}\n\nQuestion: {ex['question']}",
                      " ||| ".join(ex["answers"]["text"]) or "no answer")
metric = exact_match       # дополнительно: squad_f1
MAX_NEW_TOKENS = 32
RANKS = [4, 8, 16]
```

2.14 Вариант 14

Дано: вопрос-ответ. Вопросы, на часть которых в тексте нет ответа (эталон `no answer`); отдельно посчитайте долю верных `no answer`. Метрика: F1, «нет ответа». Ряд рангов: 8, 16, 32.

```
ds = load_dataset("rajpurkar/squad_v2")
train_raw, test_raw = ds["train"], ds["validation"]
to_pair = lambda ex: ("Answer the question using the context. If there is no answer, say
    'no answer'.\n\n"
                      f"Context: {ex['context']}\n\nQuestion: {ex['question']}",
                      " ||| ".join(ex["answers"]["text"]) or "no answer")
metric = squad_f1
MAX_NEW_TOKENS = 32
RANKS = [8, 16, 32]
```

2.15 Вариант 15

Дано: вопрос-ответ. Только вопросы с числовым ответом (даты, количества). Метрика: F1. Ряд рангов: 16, 32, 64.

```
ds = load_dataset("rajpurkar/squad")
ds = ds.filter(lambda ex: any(ch.isdigit() for ch in ex["answers"]["text"][0]))   #
    числовые ответы
train_raw, test_raw = ds["train"], ds["validation"]
to_pair = lambda ex: ("Answer the question using the context. If there is no answer, say
    'no answer'.\n\n"
                      f"Context: {ex['context']}\n\nQuestion: {ex['question']}",
                      " ||| ".join(ex["answers"]["text"]) or "no answer")
metric = squad_f1
MAX_NEW_TOKENS = 32
RANKS = [16, 32, 64]
```

2.16 Вариант 16

Дано: перевод. Перевод с английского на немецкий. Метрика: BLEU. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("Helsinki-NLP/opus-100", "de-en")
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Translate from English to German.\n\n" + ex["translation"]["en"],
    ex["translation"]["de"])
metric = bleu
MAX_NEW_TOKENS = 96
RANKS = [4, 8, 16]
```

2.17 Вариант 17

Дано: перевод. Перевод с английского на французский. Метрика: BLEU. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("Helsinki-NLP/opus-100", "en-fr")
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Translate from English to French.\n\n" + ex["translation"]["en"],
    ex["translation"]["fr"])
metric = bleu
MAX_NEW_TOKENS = 96
RANKS = [8, 16, 32]
```

2.18 Вариант 18

Дано: перефразирование. Перефразирование вопроса Quora; метрика — косинусная близость эмбеддингов ответа и эталона (all-MiniLM-L6-v2). Метрика: Сем. близость. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("nyu-mll/glue", "qqp")          # Quora Question Pairs
train_raw = ds["train"].filter(lambda ex: ex["label"] == 1)       # только пары
    дубликатов
test_raw = ds["validation"].filter(lambda ex: ex["label"] == 1)
to_pair = lambda ex: ("Paraphrase the question.\n\n" + ex["question1"], ex["question2"])
metric = semantic_similarity
MAX_NEW_TOKENS = 48
RANKS = [4, 8, 16]
```

2.19 Вариант 19

Дано: перефразирование. Перефразирование вопроса Quora; разнообразие (Distinct-2) и близость к эталону (ROUGE-L). Метрика: Distinct-2, ROUGE-L. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("nyu-mll/glue", "qqp")          # Quora Question Pairs
train_raw = ds["train"].filter(lambda ex: ex["label"] == 1)       # только пары
    дубликатов
test_raw = ds["validation"].filter(lambda ex: ex["label"] == 1)
to_pair = lambda ex: ("Paraphrase the question.\n\n" + ex["question1"], ex["question2"])
metric = distinct2_metric     # дополнительно: rouge("rougeL")
MAX_NEW_TOKENS = 48
RANKS = [8, 16, 32]
```

2.20 Вариант 20

Дано: перевод. Перевод с английского на русский; дополнительно — отношение длины перевода к эталону. Метрика: BLEU, длина. Ряд рангов: 16, 32, 64.

```
ds = load_dataset("Helsinki-NLP/opus-100", "en-ru")
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Translate from English to Russian.\n\n" +
    ex["translation"]["en"], ex["translation"]["ru"])
metric = bleu                        # дополнительно: length_ratio(P, R)
MAX_NEW_TOKENS = 96
RANKS = [16, 32, 64]
```

2.21 Вариант 21

Дано: классификация генерацией. Модель отвечает названием темы новости (4 метки). Метрика: Точность метки. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("fancyzhx/ag_news")
train_raw, test_raw = ds["train"], ds["test"]
LABELS = ds["train"].features["label"].names
to_pair = lambda ex: (f"Classify the text. Answer with one label from: {',
    '.join(LABELS)}.\n\n{ex['text']}",
                      LABELS[ex["label"]])
metric = label_accuracy(LABELS)
MAX_NEW_TOKENS = 8
RANKS = [4, 8, 16]
```

2.22 Вариант 22

Дано: классификация генерацией. Модель отвечает названием эмоции (6 меток). Метрика: Точность метки. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("dair-ai/emotion")
train_raw, test_raw = ds["train"], ds["test"]
LABELS = ds["train"].features["label"].names
to_pair = lambda ex: (f"Classify the text. Answer with one label from: {',
    '.join(LABELS)}.\n\n{ex['text']}",
                      LABELS[ex["label"]])
metric = label_accuracy(LABELS)
MAX_NEW_TOKENS = 8
RANKS = [8, 16, 32]
```

2.23 Вариант 23

Дано: классификация генерацией. Модель отвечает тональностью твита (3 метки). Метрика: Точность метки. Ряд рангов: 4, 8, 16.

```
ds = load_dataset("cardiffnlp/tweet_eval", "sentiment")
train_raw, test_raw = ds["train"], ds["test"]
LABELS = ds["train"].features["label"].names
to_pair = lambda ex: (f"Classify the text. Answer with one label from: {',
    '.join(LABELS)}.\n\n{ex['text']}",
                      LABELS[ex["label"]])
metric = label_accuracy(LABELS)
MAX_NEW_TOKENS = 8
RANKS = [4, 8, 16]
```

2.24 Вариант 24

Дано: рассуждение. Решение школьной задачи с ответом в конце в виде `#### N`; основная метрика — доля ответов в этом формате, дополнительная — точность числа (у модели 1,1 млрд параметров ожидаемо низкая). Метрика: Формат, точность. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("openai/gsm8k", "main")
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Solve the problem step by step. End with '#### <number>'.\n\n" +
    ex["question"], ex["answer"])
metric = gsm_format               # дополнительно: gsm_accuracy
MAX_NEW_TOKENS = 256
RANKS = [8, 16, 32]
```

2.25 Вариант 25

Дано: структурный вывод. Извлечение города и даты в JSON (как в примере шага 1.2). Метрика: Точность полей. Ряд рангов: 4, 8, 16.

```
# make_extraction_dataset -- приложение А
sp = make_extraction_dataset().train_test_split(test_size=0.1, seed=SEED)
train_raw, test_raw = sp["train"], sp["test"]
to_pair = lambda ex: (ex["prompt"], ex["response"])
metric = json_fields
MAX_NEW_TOKENS = 32
RANKS = [4, 8, 16]
```

2.26 Вариант 26

Дано: медицина. Ответ yes/no/maybe на вопрос по аннотации статьи (обрезается до 512 токенов); всего 1000 примеров. Метрика: Точность. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("qiaojin/PubMedQA", "pqa_labeled")["train"]      # всего 1000 примеров
sp = ds.train_test_split(test_size=0.1, seed=SEED)
train_raw, test_raw = sp["train"], sp["test"]                      # N_TRAIN фактически
    800
to_pair = lambda ex: ("Answer yes, no or maybe.\n\nContext: " + cut("
    ".join(ex["context"]["contexts"]))
                      + "\n\nQuestion: " + ex["question"], ex["final_decision"])
metric = label_accuracy(["yes", "no", "maybe"])
MAX_NEW_TOKENS = 4
RANKS = [8, 16, 32]
```

2.27 Вариант 27

Дано: финансы. Модель отвечает тональностью финансовой новости (3 метки). Метрика: Точность метки. Ряд рангов: 4, 8, 16.

```
# зеркало financial_phrasebank (sentences_allagree): оригинал загружается только скриптом
ds = load_dataset("gtfintechlab/financial_phrasebank_sentences_allagree", "5768")
train_raw, test_raw = ds["train"], ds["test"]
LABELS = ["negative", "neutral", "positive"]
to_pair = lambda ex: (f"Classify the text. Answer with one label from: {',
    '.join(LABELS)}.\n\n{ex['sentence']}",
                      LABELS[ex["label"]])
metric = label_accuracy(LABELS)
MAX_NEW_TOKENS = 8
RANKS = [4, 8, 16]
```

2.28 Вариант 28

Дано: стилизация. Инструкция начинается с «Answer briefly.» (эталон не длиннее 40 слов) или «Answer in detail.»; метрика — доля ответов нужной длины. Метрика: Соблюдение стиля. Ряд рангов: 8, 16, 32.

```
ds = load_dataset("databricks/databricks-dolly-15k")["train"]
sp = ds.train_test_split(test_size=0.1, seed=SEED)
train_raw, test_raw = sp["train"], sp["test"]



def to_pair(ex):
    style = "Answer briefly." if len(ex["response"].split()) <= 40 else "Answer in
    detail."
    text = ex["instruction"] + ("\n\n" + ex["context"] if ex["context"] else "")
    return (style + " " + text, ex["response"])



metric = style_ok
MAX_NEW_TOKENS = 256
RANKS = [8, 16, 32]
```

2.29 Вариант 29

Дано: структурный вывод. Извлечение получателя и суммы платежа в JSON. Метрика: Точность полей. Ряд рангов: 4, 8, 16.

```
# make_payment_dataset -- приложение A
sp = make_payment_dataset().train_test_split(test_size=0.1, seed=SEED)
train_raw, test_raw = sp["train"], sp["test"]
to_pair = lambda ex: (ex["prompt"], ex["response"])
metric = json_fields
MAX_NEW_TOKENS = 32
RANKS = [4, 8, 16]
```

2.30 Вариант 30

Дано: суммаризация. Краткое изложение бытового диалога (DialogSum); четыре ранга. Метрика: ROUGE-L. Ряд рангов: 8, 16, 32, 64.

```
ds = load_dataset("knkarthick/dialogsum")
train_raw, test_raw = ds["train"], ds["test"]
to_pair = lambda ex: ("Summarize the dialogue.\n\n" + ex["dialogue"], ex["summary"])
metric = rouge("rougeL")
MAX_NEW_TOKENS = 96
RANKS = [8, 16, 32, 64]
```

3 Приложения

Приложение А. Генераторы синтетических данных (варианты 25, 29)

```
import random
from datasets import Dataset


MONTHS = ["января", "февраля", "марта", "апреля", "мая", "июня",
          "июля", "августа", "сентября", "октября", "ноября", "декабря"]


def make_extraction_dataset(n=2000, seed=1025):
    """ЛР2, вариант 25: извлечение города и даты формат( примера: Казань, 12.05)."""
    rng = random.Random(seed)
    cities = ["Казань", "Москва", "Сочи", "Пермь", "Тверь", "Омск", "Курск", "Самара",
    "Томск", "Уфа"]
    T = ["рейс прилетел в {c} {d} {m}", "поезд прибывает в {c} {d} {m}",
```

```
         "встреча запланирована в {c} на {d} {m}", "доставка в {c} ожидается {d} {m}",
         "{d} {m} состоится концерт, город {c}", "конференция пройдёт {d} {m} в {c}"]
    prompts, responses = [], []
    for _ in range(n):
        c, d, m = rng.choice(cities), rng.randint(1, 28), rng.randint(1, 12)
        text = rng.choice(T).replace("{c}", c).replace("{d}", str(d)).replace("{m}",
    MONTHS[m - 1])
        prompts.append(f"Извлеки город и дату в JSON: {text}")
        responses.append(f'город{{"": "{c}", дата"": "{d:02d}.{m:02d}"}}')
    return Dataset.from_dict({"prompt": prompts, "response": responses})


def make_payment_dataset(n=2000, seed=1029):
    """ЛР2, вариант 29: извлечение получателя и суммы из текста платежа в JSON."""
    rng = random.Random(seed)
    names = ["Иван Петров", "Анна Смирнова", "Олег Кузнецов", "Мария Волкова", "Сергей
    Орлов",
             "Елена Соколова", "Дмитрий Лебедев", "Ольга Новикова"]
    T = ["переведи {s} рублей получателю {p}", "оплати {p} сумму {s} руб.",
         "нужно отправить {p} {s} ₽", "платёж на имя {p}: {s} рублей",
         "{p} должен получить {s} руб.", "перечисли {s} ₽ на счёт {p}"]
    prompts, responses = [], []
    for _ in range(n):
        p, s = rng.choice(names), rng.randint(100, 99999)
        text = rng.choice(T).replace("{p}", p).replace("{s}", str(s))
        prompts.append(f"Извлеки получателя и сумму в JSON: {text}")
        responses.append(f'получатель{{"": "{p}", сумма"": {s}}}')
    return Dataset.from_dict({"prompt": prompts, "response": responses})
```

Приложение Б. Метрики задач

Все метрики принимают список ответов модели P и список эталонов R и возвращают число; чем больше, тем лучше. В вопросно-ответных вариантах эталон может содержать несколько допустимых ответов через || ".

```
import string
from collections import Counter
from rouge_score import rouge_scorer
import sacrebleu


def rouge(kind):                                    # kind: "rouge1", "rouge2", "rougeL"
    sc = rouge_scorer.RougeScorer([kind], use_stemmer=True)
    return lambda P, R: float(np.mean([sc.score(r, p)[kind].fmeasure for p, r in zip(P,
    R)]))
```

```
def bleu(P, R):
    return sacrebleu.corpus_bleu(P, [R]).score


def _norm(s):
    s = "".join(ch for ch in s.lower() if ch not in set(string.punctuation))
    return " ".join(re.sub(r"\b(a|an|the)\b", " ", s).split())


def _f1(p, r):
    p, r = _norm(p).split(), _norm(r).split()
    c = sum((Counter(p) & Counter(r)).values())
    if not p or not r:
        return float(p == r)
    return 0.0 if c == 0 else 2 * c / (len(p) + len(r))


def squad_f1(P, R):                                  # R: варианты ответа через " ||| "
    return float(np.mean([max(_f1(p, x) for x in r.split(" ||| ")) for p, r in zip(P,
    R)]))


def exact_match(P, R):
    return float(np.mean([any(_norm(p) == _norm(x) for x in r.split(" ||| ")) for p, r
    in zip(P, R)]))


def label_accuracy(labels):
    """Ответ засчитывается, если в нём ровно одна метка из списка и она верная."""
    def f(P, R):
        def parse(p):
            hits = [l for l in labels if re.search(r"\b" + re.escape(l.lower()) + r"\b",
    p.lower())]
            return hits[0].lower() if len(hits) == 1 else None
        return float(np.mean([parse(p) == r.lower() for p, r in zip(P, R)]))
    return f


def _entities(s):                                    # "PER: A, B; LOC: C" ->
    {("PER","a"), ...}
    out = set()
    for part in s.split(";"):
        if ":" in part:
            t, xs = part.split(":", 1)
```

```
            out |= {(t.strip().upper(), x.strip().lower()) for x in xs.split(",") if
    x.strip()}
    return out


def entity_prf(P, R):
    tp = fp = fn = 0
    for p, r in zip(P, R):
        a, b = _entities(p), _entities(r)
        tp += len(a & b); fp += len(a - b); fn += len(b - a)
    prec, rec = tp / max(1, tp + fp), tp / max(1, tp + fn)
    return {"precision": prec, "recall": rec, "f1": 2 * tp / max(1, 2 * tp + fp + fn)}


def json_fields(P, R):
    """Доля верно извлечённых полей JSON невалидный( JSON -- все поля неверны)."""
    def parse(s):
        try:
            d = json.loads(s.strip().split("\n")[0])
            return d if isinstance(d, dict) else {}
        except ValueError:
            return {}
    return float(np.mean([np.mean([parse(p).get(k) == v for k, v in parse(r).items()])
                          for p, r in zip(P, R)]))


def length_ratio(P, R):
    return float(np.mean([len(p.split()) / max(1, len(r.split())) for p, r in zip(P,
    R)]))


def style_ok(P, R):
    """Коротко«» -- не более 40 слов, подробно«» -- больше 40; нужный стиль задан
    эталоном."""
    return float(np.mean([(len(p.split()) <= 40) == (len(r.split()) <= 40) for p, r in
    zip(P, R)]))


def _last_number(s):
    m = re.findall(r"-?\d[\d,]*\.?\d*", s.replace("$", ""))
    return m[-1].replace(",", "").rstrip(".") if m else None


def gsm_format(P, R):
    return float(np.mean([bool(re.search(r"####\s*-?[\d,.]+", p)) for p in P]))
```

```python
def gsm_accuracy(P, R):
    return float(np.mean([_last_number(p) == _last_number(r) for p, r in zip(P, R)]))


def semantic_similarity(P, R):
    from sentence_transformers import SentenceTransformer
    st = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
    a = st.encode(P, normalize_embeddings=True); b = st.encode(R,
    normalize_embeddings=True)
    return float(np.mean((a * b).sum(-1)))


def distinct2(texts):
    """Доля уникальных биграмм слов среди всех биграмм."""
    bg = [tuple(t.split()[i:i + 2]) for t in texts for i in range(len(t.split()) - 1)]
    return len(set(bg)) / max(1, len(bg))


def distinct2_metric(P, R):
    return distinct2(P)
```

Приложение В. Версии библиотек

Код задания проверен со следующими версиями (`requirements.txt`); с другими версиями возможны расхождения в именах аргументов:

```
torch==2.11.0
transformers==5.17.0
datasets==5.0.1
accelerate==1.15.0
scikit-learn==1.9.1
numpy==2.5.3
matplotlib==3.11.2
peft==0.21.0
trl==1.14.0
bitsandbytes==0.50.2
rouge-score==0.1.2
sacrebleu==2.6.0
sentence-transformers==6.1.0
```